

QUESTIONS

Q1

Student Grade Book Application
10 marks



Q2

Debugging and Rewriting
10 marks



2/2 answered

Q1 Student Grade Book Application

10 marks

You are building a student grade book application. Write a Python program that:

- Stores each student's name and marks in a dictionary (key = name, value = list of 5 subject marks).
- Computes total, average, and grade (O: ≥ 90 , A+: ≥ 80 , A: ≥ 70 , B+: ≥ 60 , B: ≥ 50 , F: < 50) for each student.
- Finds the class topper (highest average) and the subject with the highest overall mean.
- Displays a formatted result table sorted by average in descending order.

Use dictionary comprehension and list operations. Minimum 5 students.

Write complete, syntactically correct Python programs using lists, tuples, dictionaries, and sets with proper indentation.

Your Code

```
1 def calculate_grade(average):
2     if average >= 90:
3         return "O"
4     elif average >= 80:
5         return "A+"
6     elif average >= 70:
7         return "A"
8     elif average >= 60:
9         return "B+"
10    elif average >= 50:
11        return "B"
12    else:
13        return "F"
14
15 # (a) Dictionary storing student name as key and list of 5 subject marks
16 students = {
17     "Arun": [85, 90, 78, 92, 88],
18     "Bala": [45, 60, 55, 70, 40],
19     "Chitra": [95, 92, 89, 96, 91],
20     "Divya": [70, 65, 72, 68, 75],
21     "Elango": [55, 58, 62, 50, 60]
22 }
23
24 # (b) Computing total, average, and grade using dictionary comprehension
25 results = {
26     name: {
27         "marks": marks,
```

Input

stdin input

Output

Total	Average	Grade
=====		
=====		
=====		
Chitra	[95, 92, 89, 96, 91]	
463	92.6	O
Arun	[85, 90, 78, 92, 88]	

QUESTIONS

Q1

Student Grade Book
Application
10 marks

Q2

Debugging and Rewriting
10 marks

2/2 answered

Q2 Debugging and Rewriting

10 marks

The program below simulates a simple banking system using lists and dictionaries. It has FOUR bugs. Identify each bug with reason and rewrite the correct program.

```
accounts = {}  
def create_account(acc_no, name, balance):  
    accounts[acc_no] = {'name': name, 'balance': balance, 'txns': []}  
def deposit(acc_no, amount):  
    if acc_no in accounts:  
        accounts[acc_no]['balance'] =+ amount  
        accounts[acc_no]['txns'].append(('Deposit', amount))  
def withdraw(acc_no, amount):  
    if accounts[acc_no]['balance'] > amount:  
        accounts[acc_no]['balance'] -= amount  
        accounts[acc_no]['txns'].append(('Withdraw', amount))  
    else:  
        print('Insufficient balance')  
def mini_statement(acc_no):  
    for t in accounts[acc_no]['txns'][-5]:  
        print(t)  
create_account(1001, 'Ravi', 10000)  
deposit(1001, 5000)  
withdraw(1001, 20000)  
withdraw(1001, 3000)  
mini_statement(1001)  
print('Balance:', accounts[1001]['balance'])
```

For debugging questions: clearly state the error, the line number, the reason, and the corrected line.

Your Code

```
1 accounts = {}  
2  
3 def create_account(acc_no, name, balance):  
4     accounts[acc_no] = {'name': name, 'balance': balance, 'txns': []}  
5  
6 def deposit(acc_no, amount):  
7     if acc_no in accounts:  
8         accounts[acc_no]['balance'] += amount  
9         accounts[acc_no]['txns'].append(('Deposit', amount))  
10  
11 def withdraw(acc_no, amount):  
12     if acc_no in accounts:  
13         if accounts[acc_no]['balance'] > amount:  
14             accounts[acc_no]['balance'] -= amount  
15             accounts[acc_no]['txns'].append(('Withdraw', amount))  
16         else:  
17             print('Insufficient balance')  
18  
19 def mini_statement(acc_no):  
20     if acc_no in accounts:  
21         for t in accounts[acc_no]['txns'][-5]:  
22             print(t)  
23  
24 create_account(1001, 'Ravi', 10000)  
25 deposit(1001, 5000)  
26 withdraw(1001, 20000)  
27 withdraw(1001, 3000)
```

Input

stdin input

Output

```
Insufficient balance  
(('Deposit', 5000)  
(('Withdraw', 3000)  
Balance: 12000
```